CH-8 PART - 2

In-Depth Study Notes: Advanced Date Arithmetic and Strict Rules for Set Operators

As an expert Oracle SQL Developer, I have analyzed the execution logs in the newly provided source file (CH8-3.TXT). These logs transition from basic UNION concepts into **Granular Tenure Calculation**, **Strict Set Operator Syntax Rules**, and the complex interactions between **Pagination (FETCH/OFFSET) and Compound Queries**.

Concept 1: Granular Tenure & Age Calculation (MONTHS_BETWEEN, ADD_MONTHS, MOD)

Core Purpose: While standard date math can find the total elapsed days, businesses frequently require highly accurate breakdowns of tenure into specific **Years, Months, and Days** (e.g., calculating pension eligibility or exact age). This requires chaining multiple specialized date functions to extract the remainders.

Keyword Breakdown:

- **MONTHS_BETWEEN(newer_date, older_date)**: Calculates the highly accurate fractional elapsed months 1.
- **TRUNC**: Violently strips the decimals off the elapsed months to give whole, completed years or months 1.
- **MOD(dividend, divisor)**: The modulo operator. It divides the total elapsed months by 12 and extracts strictly the remainder (the "extra" months that don't make up a full year) 1.
- **ADD_MONTHS**: Used here dynamically to shift the hire date forward by the exact number of *completed* months, allowing us to subtract it from SYSDATE to find the leftover days 1.

Comparison of Date Remainder Functions:

Calculation Target,Oracle SQL Formula,Execution Behavior
Total Years,"TRUNC(MONTHS_BETWEEN(d1, d2) / 12)",Drops fractional remainders to output whole years 1.
Extra Months,"MOD(TRUNC(MONTHS_BETWEEN(d1, d2)), 12)","Divides elapsed months by 12, outputs the remainder 2."
Extra Days,"TRUNC(d1 - ADD_MONTHS(d2, total_months))","Shifts the start date forward, subtracts to find leftover days 2."

Syntax Breakdown:

```
SELECT
    TRUNC(MONTHS_BETWEEN(end_date, start_date)/12) AS YEARS,
    MOD(TRUNC(MONTHS_BETWEEN(end_date, start_date)), 12) AS MONTHS,
    TRUNC(end_date - ADD_MONTHS(start_date,
TRUNC(MONTHS_BETWEEN(end_date, start_date)))) AS DAYS
FROM table_name;
```

Detailed Example:HR requires a granular report showing the absolute exact tenure of employees formatted into "Years Completed", "Extra Months Completed", and "Extra Days Completed".

```
SELECT LAST_NAME, SYSDATE, HIRE_DATE,
    TRUNC(MONTHS_BETWEEN(SYSDATE, HIRE_DATE)/12) AS "TOTAL YEARS
COMPLETED",
    MOD(TRUNC(MONTHS_BETWEEN(SYSDATE, HIRE_DATE)), 12) AS "EXTRA
MONTHS COMPLETED",
    TRUNC(SYSDATE - ADD_MONTHS(HIRE_DATE,
TRUNC(MONTHS_BETWEEN(SYSDATE, HIRE_DATE)))) AS "EXTRA DAYS COMPLETED"
FROM EMPLOYEES;
```

[Execution Output: King has completed 38 Years, 10 Months, and 4 Days of service 2, 3.]

Execution Logic: The CPU evaluates from the inside out. It calculates the exact fractional months, truncates it to an integer, performs the division or modulo arithmetic, and then executes the outer standard arithmetic subtraction for the raw days.

Pro-Tips/Best Practices:

- **Always put SYSDATE first:** The MONTHS_BETWEEN function requires the newer date as the first parameter. Reversing them will result in negative numbers, completely breaking the MOD and TRUNC calculations.

Concept 2: Strict Architectural Rules for Set Operators (UNION ALL)

Core Purpose: Set Operators (UNION, UNION ALL, MINUS, INTERSECT) stack query results vertically. However, the Oracle Engine enforces strict validation checks before allowing two different tables to be combined.

Keyword Breakdown:

- **UNION ALL**: Combines datasets vertically without deduplicating or sorting, offering maximum performance 4.
- **TO_CHAR**: Used heavily in compound queries to force numeric or date columns into matching VARCHAR2 string types 5, 6.
- **NULL**: Used as an empty placeholder to force column counts to match when joining a wide table with a narrow table 7.

Comparison of Set Operator Failure Triggers:

Error Code,Trigger Condition,Resolution Strategy
ORA-01789,Query blocks have an incorrect number of result columns 8.,"Inject NULL or dummy literals (like 420) into the smaller query to match lengths 7, 8."
ORA-01790,Expressions must have the same datatype as corresponding expressions 5.,"Wrap mismatched columns in explicit conversion functions like TO_CHAR() 5, 6."
Alias Ignored,Attempting to alias the bottom query's columns.,"None. Oracle strictly uses the column aliases defined in the first (top) query 9, 10."

Syntax Breakdown:

```
SELECT TO_CHAR(date_col) AS "Dynamic Heading", TO_CHAR(number_col)
FROM table_a
UNION ALL
SELECT string_col, NULL
```

```
FROM table_b;
```
Detailed Example: An administrator needs to combine EMPLOYEES and DEPARTMENTS into a single report, but the tables have completely different structures and data types.
```
SELECT TO_CHAR(HIRE_DATE) AS "HD AND DID", TO_CHAR(SALARY) AS "SAL
AND DNM"
FROM EMPLOYEES
UNION
SELECT TO_CHAR(DEPARTMENT_ID), DEPARTMENT_NAME
FROM DEPARTMENTS;
```
[Execution: Oracle cleanly combines the dates/IDs into one string column, and the salaries/names into another string column 10.]
Execution Logic: During the parse phase, the Oracle Engine verifies rule compliance:

1. It counts the SELECT items in Query A and Query B 4.
2. It verifies the data types of Column 1 in both queries align, then Column 2, etc. 4.
3. It assigns the column headers based *exclusively* on the first query block 4, 9.If any check fails, the query immediately crashes before execution begins.

Concept 3: Localized Pagination and Sorting in Compound Queries

Core Purpose: Applying ORDER BY or OFFSET/FETCH to a compound statement can be tricky. Developers must explicitly instruct the database whether they want to sort/limit the *individual sub-components* before combining them, or sort the *final global output* after the combination.
Keyword Breakdown:

- **() (Parentheses)**: Mandatory for encapsulating an individual SELECT block if it utilizes its own sorting or limiting clauses 11, 12.
- **Positional ORDER BY**: Using integers (like ORDER BY 2) instead of column names. Essential when globally sorting compound queries that use the * meta-character 13, 14.

Comparison of Pagination/Sorting Syntax:
Intended Behavior,SQL Syntax Structure,Execution State
Limit an Individual Block,(SELECT ... FETCH FIRST 5) UNION ALL (SELECT ...),SUCCESS 12. Parentheses isolate the command.
Limit an Individual Block (No Parentheses),SELECT ... FETCH FIRST 5 UNION ALL SELECT ...,"ORA-00933 11, 15. SQL command not properly ended."
Sort an Individual Block,(SELECT ... ORDER BY col OFFSET 0 ROWS) UNION ALL ...,"SUCCESS 13, 16. OFFSET or FETCH is mandatory inside the parentheses."
Global Sort with SELECT *,"SELECT * FROM A UNION ALL SELECT ID, NM FROM B ORDER BY NM","ORA-00904 13, 17. Invalid identifier."
Syntax Breakdown:
```
(SELECT * FROM table_a ORDER BY col_a OFFSET 0 ROWS)
UNION ALL
(SELECT * FROM table_b ORDER BY col_b DESC FETCH FIRST 10 ROWS ONLY);
```
Detailed Example: The business wants a single report combining the 4 newest employees with the 8 oldest employees.

```
(SELECT * FROM EMPLOYEES OFFSET (SELECT COUNT(*) FROM EMPLOYEES) - 8
ROWS)
UNION ALL
(SELECT * FROM EMPLOYEES FETCH FIRST 4 ROWS ONLY);
```

[Execution: Isolates the pagination to the respective query blocks, executing them independently before merging the final 12 rows vertically 18.]

Pro-Tips/Best Practices:

- **The ORDER BY Workaround inside Parentheses:** Oracle will throw an ORA-00907: missing right parenthesis if you simply put ORDER BY inside a set operator subquery block 16. **Rule 5 dictates** that you *must* pair the ORDER BY with either an OFFSET or FETCH clause. To bypass this and sort the block without losing data, use a dummy limit: ORDER BY SALARY OFFSET 0 ROWS 13, 16.
- **Positional Sorting is Mandatory for *:** If you write SELECT * FROM REGIONS UNION ALL SELECT DEPT_ID, DEPT_NAME FROM DEPARTMENTS ORDER BY DEPARTMENT_NAME, the engine crashes because the REGIONS table does not have a DEPARTMENT_NAME column 13. You must use positional sorting based on the first query block: ORDER BY 2 14, 19.